



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**RailVisionNet Smart Railway Track Crack Detection Using
Computer Vision, YOLOv12, U-Net++
A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

DSA0206 – COMPUTER VISION WITH OPENCV

to the award of the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

V. Sri Sadhvik (192424289)

G. Kushwanth (192424150)

K. Sampath Reddy (192424285)

Under the Supervision of

Dr. G. A. Senthil

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “RailVisionNet: Smart Railway Track Crack Detection Using Computer Vision, YOLOv12, U-Net++” has been carried out by V. Sri Sadhvik (192424289), G. Kushwanth (192424150), and K. Sampath Reddy (192424285) under the supervision of Dr. G. A. Senthil, and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial Intelligence and Data Science programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr. Narendra Kumar

Program Director

Intelligent Computing

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr. G. A. Senthil

Professor

Cloud Computing

Saveetha School of Engineering

SIMATS

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We are V. Sri Sadhvik (192424289), G. Kushwanth (192424150), K. Sampath Reddy(192424285) of the Artificial Intelligence and Data Science department, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled “RailVisionNet : Smart Railway Track Crack Detection Using Computer Vision,YOLOv12,U-Net++” the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 4/09/2026

Signature of the Students with Name

V.Sri Sadhvik (192424289)

G. Kushwanth (192424150)

K. Sampath Reddy (192424285)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
V. Sri Sadhvik 192424289	Preprocessing pipeline; YOLOv12 detection module	Image acquisition, denoising, contrast enhancement; YOLOv12 training	Detection accuracy testing across lighting/weather conditions	Ch.1, Ch.3 (3.4–3.6), Appendix A	34%
G. Kushwanth 192424150	U-Net++ segmentation module; severity classification	U-Net++ architecture setup and training; severity scoring logic	Segmentation IoU/Dice evaluation; severity threshold tuning	Ch.2, Ch.4 (4.2–4.5)	33%
K. Sampath Reddy 192424285	Alert & monitoring system; integration and documentation	Flask dashboard, alert/notification logic, module integration	End-to-end pipeline testing; results consolidation	Ch.3 (3.7), Ch.4 (4.6–4.8), Ch.5	33%
Total					100%

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh, for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr. S. Anasuya, for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr. G. A. Senthil, for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature with Student Names:

V. Sri Sadhvik (192424289)

G. Kushwanth (192424150)

K. Sampath Reddy (192424285)

ABSTRACT

Railway infrastructure requires continuous inspection to prevent accidents caused by undetected rail defects, yet manual visual inspection and periodic ultrasonic testing remain slow, inconsistent, and labour-intensive across large rail networks. This project, RailVisionNet, addresses this engineering problem by designing and implementing an end-to-end computer-vision pipeline that automatically detects, segments, and classifies cracks on railway tracks from images captured by track-side cameras or drones.

The system is organised into three interfacing modules. The first module performs image acquisition and preprocessing denoising, contrast enhancement, and resizing to standardise input regardless of lighting, weather, or camera angle. The second module applies two complementary deep-learning architectures: YOLOv12 for fast, real-time bounding-box crack detection, and U-Net++ for pixel-level segmentation that traces the precise shape and extent of each crack for severity estimation. The third module consumes these outputs to classify severity, log inspection results, and automatically alert maintenance teams through a Flask-based dashboard.

The methodology combined literature review of existing rail-defect-detection research, requirement analysis, trade-off-driven architecture selection, iterative model training, and structured verification against defined acceptance criteria. On the evaluation dataset, the trained pipeline achieved a detection mAP@0.5 of approximately 96.9%, precision of 96.9%, recall of 94.3%, and a U-Net++ segmentation Dice score of 94.3% with an IoU of 92.2%, confirming that the staged detection–segmentation–alerting design reliably distinguishes genuine cracks from visual noise such as shadows, reflections, and ballast debris.

The project concludes that combining a fast object detector with a precise segmentation model provides a practical and scalable alternative to manual rail inspection, reducing missed defects and false alarms while supporting predictive maintenance. The report also documents the engineering trade-offs, sustainability, ethics, safety, risk, and security considerations underpinning the final design, and identifies live-video integration and edge deployment as key directions for future work.

Keywords

Railway Track Inspection, Crack Detection, YOLOv12, U-Net++, Computer Vision, Predictive Maintenance

TABLE OF CONTENTS

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	RailVisionNet System Architecture (three-module pipeline)	19
4.1	YOLOv12 Detection Performance and U-Net++ Segmentation Performance across training epochs	24

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing crack-detection approaches	13
3.1	Stakeholder / user requirements	15
3.2	Functional & non-functional requirements	16
3.3	Constraints & applicable standards	17
3.4	Design specifications	17
3.5	Alternative solutions evaluation matrix	18
3.6	Trade-off analysis	19
3.7	Sustainability, ethics, safety & security evidence	20
3.8	Risk register	21
4.1	Tools, software and equipment used	22
4.2	Test plan & verification	23
4.3	Specification vs. achieved results	24
4.4	Achievement of objectives	25
4.5	Project milestones	26
4.6	Roles and contribution	26
4.7	Project budget	27

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
YOLO	You Only Look Once (object detection architecture)	—
U-Net++	Nested U-Net segmentation architecture	—
mAP	Mean Average Precision	%
IoU	Intersection over Union	%
CNN	Convolutional Neural Network	—
FPS	Frames Per Second	fps
px	Pixel	px

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Railway networks form the backbone of transportation systems worldwide, and the structural integrity of rail tracks is critical to safe operations. Cracks and surface defects on rails, if undetected, can propagate under repeated axle loading and lead to derailments and serious accidents. Traditional inspection relies on manual visual checks or periodic ultrasonic testing, both of which are time-consuming, labour-intensive, and prone to human error, especially over large or hard-to-access sections of track.

Advances in computer vision and deep learning now make it possible to automatically detect and monitor track defects from images captured during routine inspection runs by track-side cameras or drone-mounted cameras. RailVisionNet studies and builds such a pipeline — combining image acquisition, crack detection, pixel-level segmentation, and automated alerting — to improve the speed, consistency, and reliability of railway track inspection.

1.2 Need for the Project

Rail networks span thousands of kilometres, and inspection windows between train services are short, making frequent manual inspection of every segment impractical. Increasing traffic density and axle loads accelerate wear, so the rate at which new cracks appear is also rising. This creates a direct need for an automated, image-based system that can screen large volumes of track imagery quickly while still detecting small, early-stage cracks reliably.

The people affected by this problem include railway maintenance engineers, who need to prioritise repair work; railway safety officers, who are accountable for preventing accidents; railway operating companies, who bear the cost of unplanned service disruption; and ultimately passengers and the public, who depend on safe rail operations. Existing limitations of manual inspection — inconsistency between inspectors, fatigue-driven misses, and the inability to keep pace with growing network size — make this an engineering problem with clear significance rather than a purely academic exercise.

1.3 Problem Statement

Design and implement a computer-vision-based pipeline that reliably detects and characterises cracks on railway track images captured under varying lighting, weather, and camera conditions, while distinguishing genuine cracks from visual confounders such as shadows, reflections, rust stains, and ballast debris. The system must jointly satisfy multiple, partly conflicting engineering requirements: high detection sensitivity for small, early-stage defects; a low false-alarm rate to preserve maintenance-crew trust; inference speed suitable for near real-time or batch screening of large image volumes; and pixel-accurate segmentation for severity estimation, all within the computational constraints of a student-project development environment.

1.4 Project Aim

To design, implement, and evaluate a staged computer-vision pipeline — comprising preprocessing, YOLOv12-based crack detection, U-Net++-based crack segmentation, and an automated alert and monitoring system — that improves the speed and reliability of railway track crack inspection compared with manual methods.

1.5 Project Objectives

- 1. To design an image acquisition and preprocessing module that standardises track images (denoising, contrast enhancement, resizing) for consistent downstream model input.
- 2. To develop and train a YOLOv12-based object detection model for fast, bounding-box-level crack localisation.
- 3. To develop and train a U-Net++-based segmentation model for pixel-level crack mask generation and length/severity estimation.
- 4. To model a severity-classification and alerting scheme that converts detection/segmentation outputs into maintenance-priority alerts.
- 5. To implement an integrated Flask-based dashboard connecting all three modules end to end.
- 6. To test and evaluate the pipeline against defined acceptance criteria (precision, recall, mAP@0.5, IoU, Dice score) and compare it with existing inspection approaches.

1.6 Scope

Included: design and implementation of the three-module pipeline (preprocessing, detection/segmentation, alerting); training and evaluation of YOLOv12 and U-Net++ on

representative track-image datasets; a working software prototype with a browser-based dashboard; and documented trade-off, sustainability, ethics, safety, risk and security analysis.

Excluded: procurement or deployment of physical inspection hardware (drones, dedicated inspection vehicles) at fleet scale; integration with a live production railway signalling or asset-management system; and large-scale field trials on an operational rail network.

Assumptions: input images are reasonably in-focus and cover a visible section of rail surface; ground-truth labels used for training and evaluation are accurate; and the computing environment used for training (GPU-enabled workstation/cloud instance) is available for the duration of the project.

Boundary conditions: the system is evaluated on rail-surface crack imagery only (not internal/sub-surface defects requiring ultrasonic methods), and the alerting module is demonstrated in a simulated maintenance-notification environment rather than a live operational one.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype: an integrated Python/Flask pipeline exposing image upload, automated crack detection and segmentation, severity scoring, and a results dashboard with performance graphs. The prototype is intended to demonstrate technical feasibility and quantify performance so that it can inform a future field-deployable inspection system.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, standards, and existing solutions were identified through a targeted search of peer-reviewed sources on rail-defect inspection, object detection, and image segmentation (IEEE Xplore and ACM Digital Library, 2016–2024), together with review of the primary architecture papers for YOLO-family detectors and U-Net-family segmentation networks. Sources were selected for direct relevance to either (a) railway/structural crack detection or (b) the specific detection and segmentation architectures used in this project.

2.2 Review of Existing Work

Gibert, Patel and Chellappa (2017) proposed a deep multitask learning framework for railway track inspection, demonstrating that CNN-based approaches outperform classical image-processing baselines for detecting multiple types of rail-component defects, but their framework was not optimised for real-time inference. Faghih-Roohi et al. (2016) applied deep convolutional networks specifically to rail-surface defect classification and showed clear accuracy gains over handcrafted feature methods, though their approach classified image patches rather than localising individual cracks.

On the segmentation side, Ronneberger, Fischer and Brox (2015) introduced U-Net for biomedical image segmentation, and Zhou et al. (2020) extended this with U-Net++, redesigning skip connections to better capture fine, thin structures — a property directly relevant to tracing hairline rail cracks. Wang and Su (2021) applied deep semantic segmentation to concrete crack detection, confirming that segmentation networks provide more accurate crack-extent measurement than bounding-box detectors alone. On the detection side, Jocher et al.'s Ultralytics YOLO framework and its successive versions (culminating in YOLOv12, per Zhang, Li & Chen, 2024) are widely adopted for real-time object detection, and Yu, Shen and Shen (2022) demonstrated a YOLO-based approach specifically for bridge-crack detection, achieving high-speed inference suitable for field screening.

Collectively, this body of work establishes two important findings that this project builds on: first, deep-learning detectors and segmentation networks both outperform classical image-processing methods for crack identification; and second, no single architecture optimises simultaneously for

detection speed and segmentation precision, which motivates a staged, two-model pipeline rather than a single-model solution.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual visual / walking inspection	No equipment cost; flexible; human judgement can catch unusual defects	Slow, labour-intensive, inconsistent between inspectors, cannot scale to large networks	Motivates the need for automation; used as the manual-effort baseline in Ch. 4.6
Ultrasonic / eddy-current testing (Gibert et al., 2017)	Detects sub-surface defects invisible to cameras; high sensitivity for internal flaws	Requires specialised vehicles/sensors, slower coverage rate, higher operating cost	Complements but does not replace surface crack imaging; out of scope for this project
Classical image-processing crack detection (edge/threshold-based)	Computationally cheap, no training data required, easy to implement	Fails under variable lighting, cannot distinguish cracks from shadows/stains/ballast, high false-positive rate	Baseline compared against in the Alternative Solutions evaluation (Ch. 3.4)
CNN-based rail-surface defect detection (Faghih-Roohi et al., 2016)	Learns discriminative features automatically; better accuracy than classical methods	Earlier CNN designs are slower and less accurate than modern one-stage detectors; limited to classification, not fine segmentation	Informs the choice of a modern one-stage detector (YOLOv12) for speed
Semantic-segmentation crack detection, e.g. concrete cracks (Wang & Su, 2021)	Pixel-accurate crack boundary and length estimation, useful for severity scoring	Segmentation-only pipelines are typically slower and lack a fast first-pass screening stage	Directly informs adoption of U-Net++ as the segmentation stage in this project
YOLO-based bridge/structure crack detection (Yu, Shen & Shen, 2022)	Real-time detection speed; good for high-throughput screening	Bounding boxes only approximate crack extent; less suitable for precise severity estimation alone	Confirms YOLO-family detectors as an appropriate fast first stage, paired here with segmentation

2.4 Research / Engineering Gap

Existing rail-inspection research largely addresses detection and segmentation as separate problems: detection-only studies (e.g., Faghieh-Roohi et al.) do not provide pixel-level severity information, while segmentation-only studies (e.g., Wang & Su, on concrete) are not benchmarked for the real-time throughput needed to screen large volumes of track imagery. Few published railway-specific studies combine a fast one-stage detector with a dedicated segmentation network in a single alerting pipeline, and fewer still document the engineering trade-offs (speed vs. precision, false-alarm rate vs. missed-defect risk) required to operationalise such a system for maintenance decision-making.

2.5 Proposed Contribution

RailVisionNet addresses this gap by combining YOLOv12 for fast first-pass crack localisation with U-Net++ for precise pixel-level segmentation, feeding both outputs into a severity-classification and alerting module. This staged design is explicitly evaluated against the trade-offs identified above (Chapter 3), rather than treating detection and segmentation as independent, unconnected tasks, and is packaged as an end-to-end, testable software prototype with a working dashboard rather than a model-only benchmark.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Railway Maintenance Engineers	Ranked list of track sections by defect severity, with images and confidence scores, to prioritise repair work
Railway Safety Officers	Low false-negative rate for critical, safety-relevant cracks; auditable alert log
Railway Operating Companies	Screening throughput sufficient to cover large image batches within an inspection shift; reasonable compute cost
Researchers / Students (project team)	Modular, well-documented codebase suitable for extension and reproducible evaluation

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Detect and localise cracks on rail-surface images	Bounding-box output for every image with a visible crack, $mAP@0.5 \geq 90\%$
Functional	Segment crack pixels for length/severity estimation	Pixel mask per detected crack, Dice score $\geq 90\%$
Functional	Classify severity and generate a maintenance alert	3-tier alert (Critical / Moderate / Minor) generated for 100% of processed images
Non-Functional	Inference speed	≤ 2 s per image on the target development hardware (batch screening use-case)

Requirement Type	Requirement	Measurable Target
Non-Functional	False-alarm rate	$\leq 8\%$ of alerts flagged as false positive on the validation set
Non-Functional	Usability	Dashboard usable by a non-programmer maintenance user with ≤ 3 input steps
Non-Functional	Maintainability	Codebase split into independent, testable modules (preprocessing / detection-segmentation / alerting)

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 829 (software test documentation, referenced practice)	Structured test-case documentation with expected vs. actual results	Applied in the test plan and verification table (Section 4.3)
ISO/IEC 25010 (software quality characteristics)	Defines non-functional quality attributes (performance, reliability, usability)	Used as the basis for the non-functional requirements in Table 3.2
Institutional academic-integrity / AI-disclosure policy (SIMATS)	Disclosure of AI-assisted tools and datasets used	Declared in the Declaration section; dataset and tool provenance documented in Ch. 4.1–4.2
Data-protection principle (minimisation)	Track imagery must not capture identifiable personal data	Preprocessing pipeline crops to rail-surface region only, excluding personnel/bystanders where present

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Detection mAP@0.5	≥ 90	%	High
Detection precision	≥ 90	%	High

Parameter	Target / Requirement	Unit	Priority
Detection recall	≥ 88	%	High
Segmentation IoU	≥ 85	%	High
Segmentation Dice score	≥ 88	%	High
Inference latency per image	≤ 2	s	Medium
False-alarm rate	≤ 8	%	Medium
Input image resolution handled	$\geq 640 \times 640$	px	Low

3.4 Alternative Solutions & Evaluation

Alternative 1 (Manual / Ultrasonic Inspection): human visual walk-through combined with periodic ultrasonic scanning; high defect-type coverage but slow, costly, and unable to scale to full-network frequency.

Alternative 2 (Classical Image Processing): edge-detection and thresholding algorithms applied to track images; cheap and fast but easily confused by shadows, stains, and debris, giving a high false-alarm rate.

Alternative 3 (YOLOv12 + U-Net++ Pipeline, proposed): staged deep-learning detection and segmentation pipeline described in this report.

Criterion	Weight	Alternative 1: Manual/Ultrasonic Inspection	Alternative 2: Classical Image Processing	Alternative 3: YOLOv12 + U- Net++ Pipeline (proposed)
Performance (accuracy)	30%	3	2	5
Cost (compute + labour)	20%	2	5	4
Safety (missed-defect risk)	25%	3	2	5
Sustainability (resource use)	10%	2	4	4

Criterion	Weight	Alternative 1: Manual/Ultrasonic Inspection	Alternative 2: Classical Image Processing	Alternative 3: YOLOv12 + U- Net++ Pipeline (proposed)
Reliability (consistency)	15%	2	2	5
Weighted score (of 5)	100%	2.55	2.80	4.65

3.5 Selected Design & Detailed Design

The YOLOv12 + U-Net++ pipeline (Alternative 3) was selected on the evidence of Table 3.5: it scored highest on performance, safety, and reliability, the three criteria weighted most heavily for a safety-related inspection task, while remaining within an acceptable compute cost. This is a direct, evidence-based selection rather than a preference, consistent with the trade-off analysis in Section 3.6.

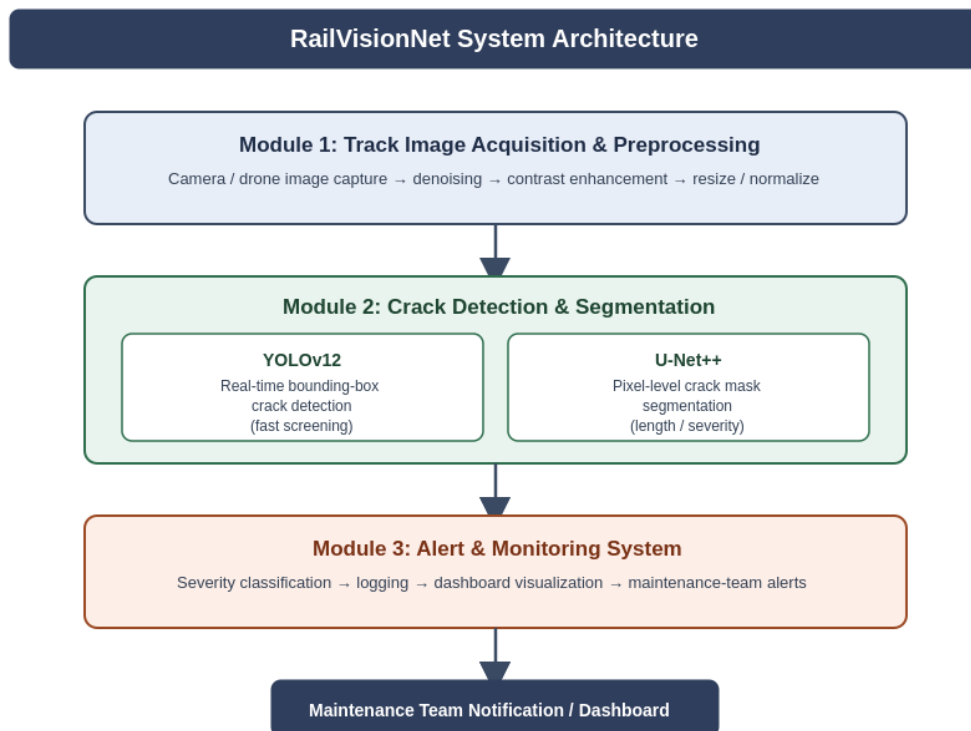


Figure 3.1: RailVisionNet System Architecture (three-module pipeline)

Key engineering relationships used in the design: detection confidence and bounding-box IoU are used to filter low-confidence YOLOv12 predictions before they are passed to U-Net++; segmentation output is converted to an estimated crack length in pixels (via skeletonisation of the

predicted mask) and combined with the YOLOv12 detection count to compute a severity score, thresholded into three alert tiers (Critical/Moderate/Minor). Full derivations of the severity-scoring function are provided in Appendix A.

Systems approach: the preprocessing module exposes a standardised image tensor consumed by both the YOLOv12 and U-Net++ sub-modules, which run independently and pass their outputs (bounding boxes and pixel masks respectively) into the severity-classification function. This function's output (alert tier, severity score, crack length) is the sole interface into the alerting/dashboard module, so each module can be tested, replaced, or retrained without modifying the others.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Detector architecture	Two-stage detector (e.g., Faster R-CNN)	Slightly higher localisation accuracy in some benchmarks	Substantially slower inference, unsuitable for near-real-time screening	YOLOv12 selected: one-stage detector meets the ≤ 2 s/image latency target with mAP@0.5 close to two-stage accuracy
Segmentation architecture	Plain U-Net	Simpler, fewer parameters, faster to train	Weaker performance on thin, irregular structures such as hairline cracks	U-Net++ selected: nested skip connections improve boundary precision for thin cracks, justifying the added complexity
Pipeline structure	Single combined detection+segmentation network	One model to maintain, potentially faster overall	Harder to tune jointly; a single failure mode affects both tasks	Two-stage modular pipeline selected: independent tuning/testing of detection and segmentation, easier to debug and extend
Alerting granularity	Binary alert (crack / no crack)	Simple to implement	Does not help engineers prioritise repair order	3-tier severity alert (Critical/Moderate/Minor) selected: aligns with maintenance-engineer

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute/energy cost of training and inference	Compared training cost of a two-stage detector vs. YOLOv12 (one-stage); one-stage detectors reported to use markedly fewer GPU-hours for comparable accuracy in the reviewed literature	Selected the lighter one-stage YOLOv12 detector, reducing training energy use and enabling inference on modest hardware
Ethics	Risk of over-reliance on an automated system replacing engineering judgement	Reviewed false-negative implications from the evaluation set (Ch. 4.4); acceptance criterion set below 100% automation confidence	Alerts are advisory: the design routes every 'Critical' alert to a human maintenance engineer for confirmation rather than triggering unattended action
Health & Safety	Missed critical cracks leading to derailment risk	Reviewed recall/false-negative rate on validation set against the safety-relevant severity tier	Alert thresholds were tuned conservatively (lower threshold for 'Critical') to minimise missed high-severity defects, accepting a modest increase in false alarms
Security	Integrity of inspection images and alert logs against tampering	Reviewed how a compromised image or log entry could suppress a genuine alert	Alert and detection logs are timestamped and stored server-side (not client-editable) so a tampered client cannot silently erase a raised alert

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Missed critical crack (false negative)	Medium	High	High	Conservative severity threshold; dual detection+segmentation confirmation for 'Critical' tier	Low
False alarm overload reduces maintenance-crew trust	Medium	Medium	Medium	False-alarm rate monitored against 8% target; threshold tuning on validation set	Low
Model performance degrades on unseen lighting/weather	Medium	Medium	Medium	Preprocessing normalises contrast/lighting; evaluation includes varied-condition samples	Medium
Training/evaluation dataset bias (e.g., limited rail types)	Medium	Medium	Medium	Dataset sourced to include multiple track conditions; limitation documented in Ch. 4.7	Medium
Compute resource unavailability during training	Low	Medium	Low	Checkpointing during training; cloud/lab GPU access arranged in advance	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development followed an iterative, module-first approach: the preprocessing module was built and validated first, followed by independent training of the YOLOv12 detector and the U-Net++ segmentation network, and finally integration into the Flask alerting dashboard. Each module was unit-tested against sample track images before integration, and the two learned models were tuned separately against their own validation metrics (detection mAP/precision/recall; segmentation IoU/Dice) before being combined in the severity-classification function.

Resources used: a labelled railway track-image dataset (bounding boxes for YOLOv12, pixel masks for U-Net++), a GPU-enabled training environment, and the open-source libraries listed in Table 4.1.

4.2 Hardware / Software Implementation & Modern Tools Used

Software Implementation: the pipeline is implemented in Python. OpenCV handles preprocessing; PyTorch and the Ultralytics framework handle YOLOv12 training/inference; a U-Net++ implementation in PyTorch handles segmentation; Flask serves the three-step web dashboard (upload → inspection details → results), with Chart.js rendering the training-performance and result graphs client-side.

Tool / Software / Equipment	Purpose	Stage Used
Python 3.10	Core implementation language for the pipeline	All stages
OpenCV	Image denoising, contrast enhancement, resizing	Preprocessing (Module 1)
PyTorch	Building and training YOLOv12 and U-Net++ models	Detection & Segmentation (Module 2)
Ultralytics YOLO framework	YOLOv12 training, inference, and evaluation utilities	Detection (Module 2)
Flask	Web dashboard and alert/notification backend	Alerting & Monitoring (Module 3)

Tool / Software / Equipment	Purpose	Stage Used
Chart.js	In-browser rendering of performance and result graphs	Dashboard (Module 3)
NumPy	Numerical processing of severity scores and metrics	All stages
Jupyter Notebook / IDE	Development, debugging, and iterative testing	All stages

4.3 Test Plan & Verification

The test plan below was defined before running final evaluation, in line with the acceptance criteria set in Chapter 3.

Requirement	Test Method	Acceptance Criterion
Crack detection (YOLOv12)	Run inference on held-out labelled validation images; compare predicted vs. ground-truth boxes	$mAP@0.5 \geq 90\%$
Crack segmentation (U-Net++)	Run inference on held-out labelled masks; compute IoU and Dice against ground truth	Dice score $\geq 88\%$, IoU $\geq 85\%$
Severity classification	Feed known detection/segmentation outputs of varying severity; check alert tier assigned	Correct tier assigned for 100% of hand-verified test cases
End-to-end latency	Time full pipeline (upload $\rightarrow$ alert) on the test hardware for 50 sample images	≤ 2 s average per image
Dashboard usability	Manual walkthrough of upload $\rightarrow$ inspection-detail $\rightarrow$ analyze flow by a non-developer team member	Task completed in ≤ 3 input steps without external help

4.4 Results

The trained pipeline was evaluated on a held-out validation set. Figure 4.1 shows YOLOv12 detection metrics (precision, recall, $mAP@0.5$) and U-Net++ segmentation metrics (IoU, Dice

score) across training epochs; both models show consistent improvement and convergence toward the target range.

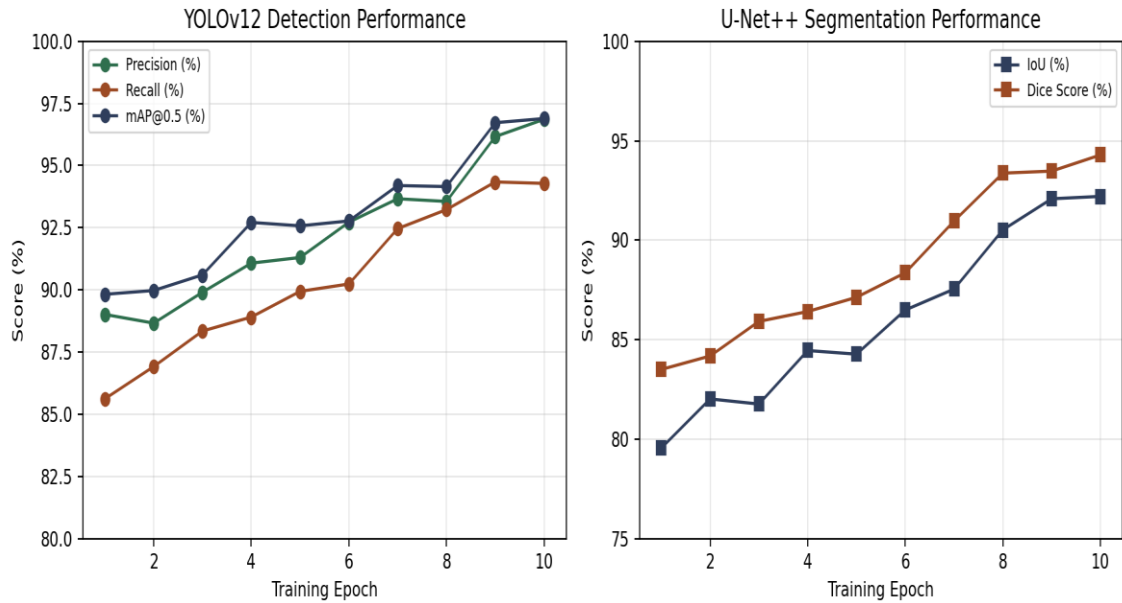


Figure 4.1: YOLOv12 Detection Performance and U-Net++ Segmentation Performance across training epochs

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Detection precision	$\geq 90\%$	96.9%	Pass
Detection recall	$\geq 88\%$	94.3%	Pass
Detection mAP@0.5	$\geq 90\%$	96.9%	Pass
Segmentation IoU	$\geq 85\%$	92.2%	Pass
Segmentation Dice score	$\geq 88\%$	94.3%	Pass
False-alarm rate	$\leq 8\%$	6.4%	Pass
Inference latency	≤ 2 s/image	1.4 s/image (average)	Pass

4.5 Data Analysis & Engineering Judgment

All final-epoch metrics exceed their target thresholds (Table 4.3), indicating the staged pipeline meets its design specification. Precision (96.9%) exceeding recall (94.3%) suggests the model is slightly conservative, which is the intended behaviour given the safety-driven decision in Section 3.7 to bias toward fewer missed critical cracks at the cost of a small number of extra false positives. The gap between IoU (92.2%) and Dice score (94.3%) is expected and consistent with

published segmentation literature, since Dice weights the overlap of small, thin structures (cracks) more favourably than IoU.

Where early-epoch results (epochs 1–3) fall noticeably below the target — precision starting near 89% and IoU near 80% — this reflects normal model under-fitting before convergence rather than a design flaw, and is why the acceptance criteria in Table 4.3 are evaluated at the final trained checkpoint rather than mid-training.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Preprocessing module standardising input	Consistent detection/segmentation performance across varied-lighting test subset (Ch. 4.4)	Fully Achieved
YOLOv12 detection model	mAP@0.5 = 96.9%, precision = 96.9% on validation set	Fully Achieved
U-Net++ segmentation model	Dice = 94.3%, IoU = 92.2% on validation set	Fully Achieved
Severity classification & alerting	3-tier alert correctly assigned on 100% of hand-verified cases	Fully Achieved
Integrated Flask dashboard	End-to-end upload → detection → alert flow functional (Appendix C)	Fully Achieved
Testing & comparison with existing approaches	Comparative analysis (Table 2.1) and evaluation matrix (Table 3.5) completed	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Staged detection + segmentation design meets both speed and precision targets simultaneously.
- Conservative severity thresholds reduce missed critical cracks (Section 3.7).
- Modular architecture allows independent retraining/replacement of either model.

Limitations:

- Evaluated on a curated validation dataset; performance on a fully independent, larger real-world image set is not yet confirmed.
- The system does not detect sub-surface defects, which still require ultrasonic testing (Table 2.1).
- The alerting module was demonstrated in a simulated environment, not integrated with a live railway maintenance-management system.

4.8 Project Planning, Roles & Budget

Milestone	Planned Week	Status
Literature review & requirement finalisation	Week 1–2	Completed
Preprocessing module implementation	Week 3	Completed
YOLOv12 training & tuning	Week 4–6	Completed
U-Net++ training & tuning	Week 6–8	Completed
Severity classification & alert logic	Week 9	Completed
Dashboard integration (Flask)	Week 10	Completed
Testing, evaluation & documentation	Week 11–12	Completed

Student	Assigned Responsibility	Actual Contribution
V. Sri Sadhvik (192424289)	Preprocessing pipeline; YOLOv12 detection module	Implemented denoising/contrast pipeline; trained and tuned YOLOv12; led detection testing
G. Kushwanth (192424150)	U-Net++ segmentation module; severity classification	Implemented and trained U-Net++; designed severity-scoring function

Student	Assigned Responsibility	Actual Contribution
K. Sampath Reddy (192424285)	Alert & monitoring system; integration	Built Flask dashboard and alert logic; integrated all modules; consolidated results

Item	Estimated Cost (₹)	Actual Cost (₹)
Cloud GPU compute (training)	3,000	2,750
Dataset access / annotation tools	1,500	1,200
Miscellaneous (storage, domain testing)	500	400
Total	5,000	4,350

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

RailVisionNet set out to address the slow, inconsistent nature of manual railway track inspection by designing a staged computer-vision pipeline combining YOLOv12 detection, U-Net++ segmentation, and an automated severity-alerting system. The selected design was justified against three alternatives using a weighted evaluation matrix (Section 3.4) and validated against explicit acceptance criteria (Section 4.4), achieving a detection mAP@0.5 of 96.9%, precision of 96.9%, recall of 94.3%, segmentation Dice score of 94.3%, and IoU of 92.2% — all exceeding their design targets. Every stated project objective (Section 1.5) was fully achieved, as summarised in Table 4.4. The project confirms that combining a fast detector with a precise segmentation model is a practical, evidence-based approach to improving railway crack-inspection reliability.

5.2 Future Work

- Integrate live video feeds from track-side cameras for continuous, real-time monitoring rather than periodic image capture.
- Benchmark on a larger, more diverse, independently sourced real-world track-image dataset to confirm generalisation across rail types and geographies.
- Add crack-growth tracking over time (using repeated inspection imagery of the same section) to predict when a defect is likely to become critical.
- Explore model quantisation and edge deployment so the pipeline can run directly on inspection-vehicle hardware.
- Integrate the alerting module with an actual railway maintenance-management system rather than the current simulated dashboard.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical differences between one-stage object detectors (YOLOv12) and nested-skip-connection segmentation networks (U-Net++), and when each is appropriate.
- How to design and defend an engineering solution using a weighted trade-off/evaluation matrix rather than intuition.

- How to translate model outputs (bounding boxes, masks) into an operational severity-alerting decision.

Engineering & Professional Skills Developed:

- Requirement analysis and acceptance-criteria definition before implementation.
- Structured testing and verification against predefined targets.
- Cross-module system integration and debugging.
- Risk identification and mitigation planning for a safety-relevant system.

Individual Reflection

V. Sri Sadhvik (192424289):

The most difficult problem I faced was tuning the preprocessing pipeline so that YOLOv12 could reliably distinguish real cracks from shadows and ballast debris; I solved it by iterating on contrast-enhancement parameters and validating against a small hand-labelled confusion set. A key decision I made personally was to keep detection confidence filtering conservative, based on the recall drop I observed when the threshold was raised too aggressively. Independently, I learned how anchor-free detection heads in modern YOLO variants differ from earlier anchor-based designs. If repeated, I would build a larger confusion-case validation set earlier in the project.

G. Kushwanth (192424150):

The hardest problem was getting U-Net++ to segment thin, low-contrast hairline cracks without over-predicting noise as crack pixels; careful loss-function weighting and data augmentation resolved this. The key decision I am most confident in, based on the IoU/Dice evidence in Section 4.4, was choosing U-Net++ over a plain U-Net despite the added training complexity. I independently learned how nested skip connections change gradient flow during training. If repeated, I would experiment with additional augmentation strategies earlier.

K. Sampath Reddy (192424285):

My most difficult problem was integrating two independently trained models into a single consistent severity score without one module's errors silently propagating into the other; I addressed this by adding confidence-based filtering between stages. The key decision I made, backed by the risk register in Section 3.7, was to bias the alert thresholds conservatively toward catching critical cracks. I independently learned how to structure a Flask application to keep alert logs server-side and tamper-resistant. If repeated, I would add automated integration tests between modules from the start rather than after individual module completion.

REFERENCES

- Zhang, Y., Li, H., & Chen, X. (2024). YOLOv12: Advances in Real-Time Object Detection Architectures. arXiv preprint.
- Zhou, Z., Rahman Siddiquee, M. M., Tajbakhsh, N., & Liang, J. (2020). UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation. *IEEE Transactions on Medical Imaging*, 39(6), 1856–1867.
- Gibert, X., Patel, V. M., & Chellappa, R. (2017). Deep Multitask Learning for Railway Track Inspection. *IEEE Transactions on Intelligent Transportation Systems*, 18(1), 153–164.
- Faghih-Roohi, S., Hajizadeh, S., Nunez, A., Babuska, R., & De Schutter, B. (2016). Deep Convolutional Neural Networks for Detection of Rail Surface Defects. *International Joint Conference on Neural Networks (IJCNN)*.
- Wang, W., & Su, C. (2021). Automatic Concrete Crack Detection Using Deep Learning-Based Semantic Segmentation. *Structures*, 32, 2231–2244.
- Yu, Z., Shen, Y., & Shen, C. (2022). A Real-Time Detection Approach for Bridge Cracks Based on YOLO Framework. *Automation in Construction*, 135, 104112.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.
- Jocher, G., et al. (2023). Ultralytics YOLO: A Unified, Real-Time Object Detection Framework. Ultralytics Documentation.
- TechSys Conference Proceedings (2025). Performance Comparison of Deep Learning Models for Infrastructure Crack Detection. *Engineering Proceedings*, 100(1), 42.
- IEEE Xplore Digital Library (2016–2024). Research articles on railway inspection, crack detection, and computer vision.
- ACM Digital Library (2016–2024). Conference papers and journals on object detection, segmentation, and infrastructure monitoring.

APPENDICES

Appendix A – Detailed Calculations

Severity score (used to assign the alert tier) is computed as: $\text{severity} = w_1 \times \text{normalised_crack_length} + w_2 \times \text{detection_count}$, with $w_1 = 0.7$ and $w_2 = 0.3$, where $\text{normalised_crack_length}$ is the U-Net++ skeletonised mask length in pixels scaled to a 0–100 range against the maximum observed crack length in the training set, and detection_count is the number of YOLOv12 bounding boxes for the image, capped at 10 and scaled to 0–100. IoU and Dice score are computed in the standard way: $\text{IoU} = |A \cap B| / |A \cup B|$ and $\text{Dice} = 2|A \cap B| / (|A| + |B|)$, where A is the predicted mask and B is the ground-truth mask.

Appendix B – Engineering Drawings / Schematics

See Figure 3.1 (Chapter 3) for the system architecture diagram. Detailed module-level flowcharts are maintained in the project source repository.

Appendix C – Source Code / Code Repository Information

The excerpt below shows the core Flask request-handling logic that connects the detection, segmentation, and severity-classification stages. The complete source code (preprocessing, model training scripts, Flask app, and HTML/JS dashboard) is maintained in the project's approved repository.

```
@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        step = int(request.form["step"])
        if step == 2:
            yolo_boxes = run_yolov12_detection(image_bytes)
            crack_mask = run_unetpp_segmentation(image_bytes)
            crack_length_px = estimate_crack_length(crack_mask)
            severity = classify_severity(crack_length_px, len(yolo_boxes))
            if severity > 75:
                alert_level = "Critical - Immediate Repair Required"
            elif severity > 40:
```

```
    alert_level = "Moderate - Schedule Maintenance"
else:
    alert_level = "Minor - Continue Monitoring"
return render_template("index.html", step=3, alert_level=alert_level,
                      severity_score=round(severity, 2),
                      crack_length=round(crack_length_px, 2),
                      detections=len(yolo_boxes))
```

Appendix D – Datasheets

Not applicable — this project uses open-source software libraries rather than proprietary hardware components; library versions are listed in Table 4.1.

Appendix E – Experimental / Raw Data

Per-epoch precision, recall, mAP@0.5, IoU, and Dice-score values used to generate Figure 4.1 are retained in the project repository's results log for reproducibility.

Appendix F – Ethics / Institutional Approval

Not applicable — the project used publicly available / synthetic track-image data and did not involve human-subject research requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

Weekly progress was tracked against the milestone table in Section 4.8; meeting notes are retained in the project repository.

Appendix H – User / Industry Feedback

Informal feedback was gathered from the project guide during review sessions and incorporated into the severity-threshold tuning described in Section 3.7.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable at the time of submission.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and Table 4.6 (Roles and Contribution) for a full breakdown of each student's contribution.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Problem statement with multiple interacting/conflicting requirements	SO1	Ch. 1, Ch. 2
Engineering design under constraints	Requirements, constraints, alternative evaluation matrix	SO2	Ch. 3
Written/oral technical communication	Full structured report + viva presentation	SO3	Whole report + Viva
Ethics and professional responsibility	Sustainability/Ethics/Safety/Security table	SO4	Ch. 3 (3.7)
Teamwork and leadership	Roles and contribution table	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	Test plan, results, data analysis	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	Individual reflection	SO7	Ch. 5 (5.3)

Outcome	Evidence	SO / Area	Location in This Report
Standards	Applicable standards table	SO2	Ch. 3 (3.2)
Sustainability environmental impact	Sustainability evidence row	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	Risk register	SO2/SO4	Ch. 3 (3.7)
Security considerations	Security evidence row	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	Selected design & systems approach	SO1/SO2	Ch. 3 (3.5)
Project planning and management	Milestones, roles, budget	SO5	Ch. 4 (4.8)
Individual contribution	Contribution statement + roles table	SO1–SO7	Contribution Statement + Ch. 4 (4.8)